



**University Of Engineering & Technology
Lahore (Narowal Campus)**

Lab Manual # 04

Compiler Construction

Submitted to:

Ms. Madiha Maqbool / Ms. Ariba

Submitted by:

Ameer Hamza

Section:

B

Roll No:

2022-CS-562

Semester:

7th (Fall 2025)

Department of Computer Science & Engineering

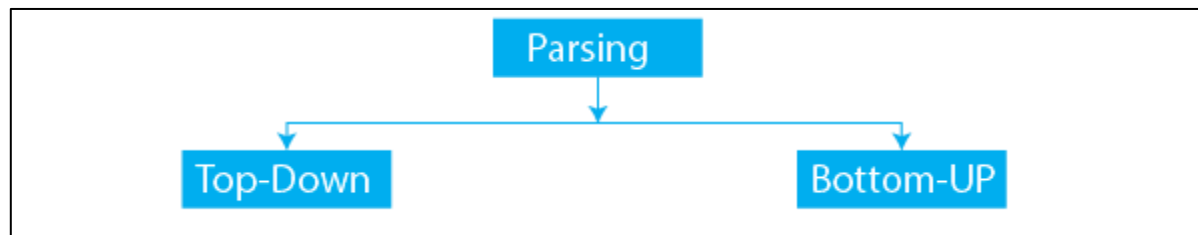
Parsing:

Parsing is the process of converting information from one type to another. The parser is a component of the translator in the organization of linear text structure according to a set of defined rules known as grammar.

Types of the Parser:

Parser is divided into two types:

- Bottom-up parser
- Top-down parser



Top-Down Parser:

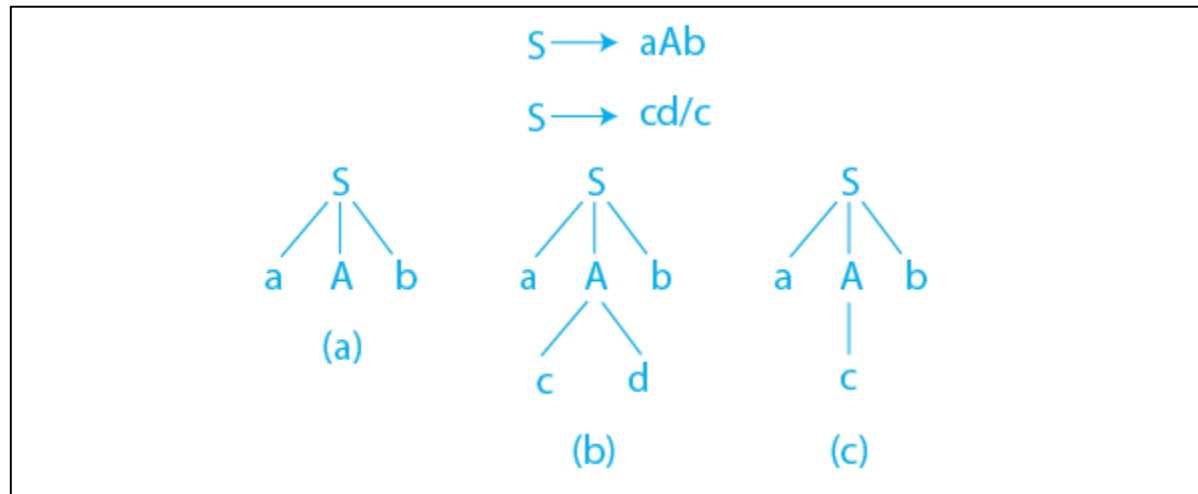
A top-down parser in compiler design can be considered to construct a parse tree for an input string in preorder, starting from the root. It can also be considered to create a leftmost derivation for an input string. The leftmost derivation is built by a top-down parser. A topdown parser builds the leftmost derivation from the grammar's start symbol. Then it chooses a suitable production rule to move the input string from left to right in sentential form.

Example of Top-Down Parsing:

Consider the lexical analyzer's input string 'acb' for the following grammar by using left leftmost deviation.

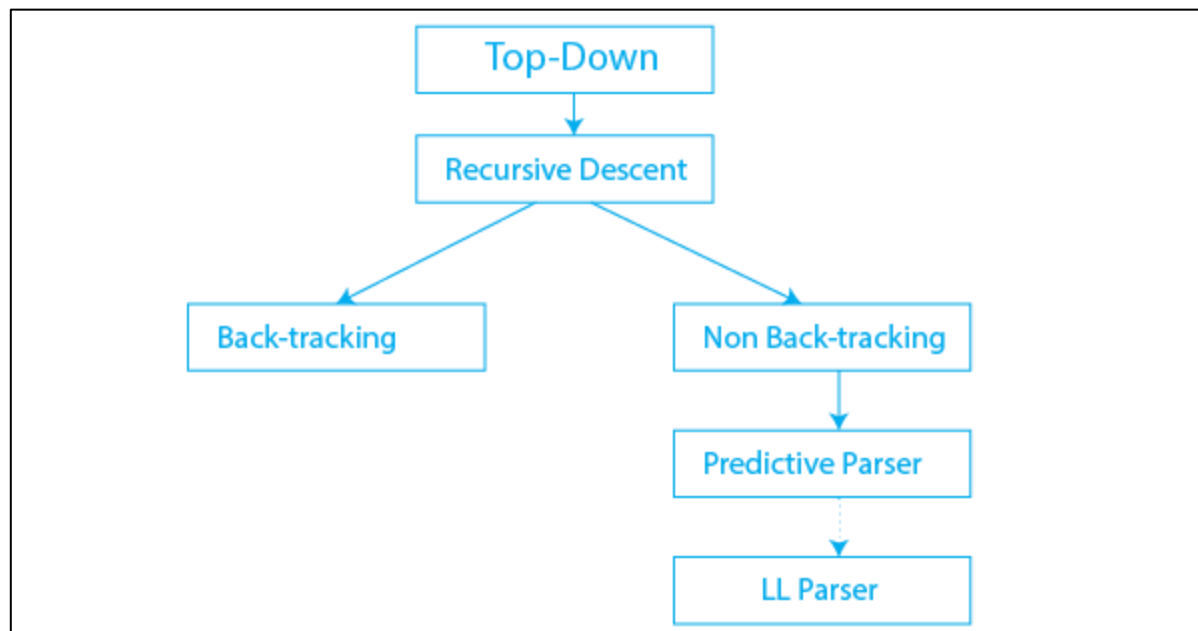
$S \rightarrow aAb$

$A \rightarrow cd \mid c$



Classification of the Top-Down Parser:

Top-down parsers can be classified based on their approach to parsing as follows:



- Recursive-descent parsers: Recursive-descent parsers are a type of top-down parser that uses a set of recursive procedures to parse the input. Each non-terminal symbol in the grammar corresponds to a procedure that parses input for that symbol.
- Backtracking parsers: Backtracking parsers are a type of top-down parser that can handle non-deterministic grammar. When a parsing decision leads to a dead end, the parser can backtrack and try another alternative.

- Non-backtracking parsers: Non-backtracking is a technique used in top-down parsing to ensure that the parser doesn't revisit already-explored paths in the parse tree during the parsing process.
- Predictive parsers: Predictive parsers are top-down parsers that use parsing to predict which production rule to apply based on the next input symbol. Predictive parsers are also called LL parsers because they construct a left-to-right, leftmost derivation of the input string.

Advantages of Top-Down Parsing:

- Top-down parsing is much simpler.
- It is incredibly easy to identify the response action of the top-down parser.

Disadvantages of Top-Down Parsing:

- Top-down parsing cannot handle left recursion in the grammar's present.
- When using recursive descent parsing, the parser may need to backtrack when it encounters a symbol that does not match the expected token. This can make the parsing process slower and less efficient.

Program:

```
1  #include <iostream>
2  #include <cctype>
3  #include <stdexcept>
4  #include <memory>
5  class TreeNode {
6  public:
7      virtual double evaluate() = 0;
8      virtual void display(int depth) = 0;
9  };
10 class BinaryOpNode : public TreeNode {
11 public:
12     BinaryOpNode(char op, std::shared_ptr<TreeNode> left, std::shared_ptr<TreeNode> right)
13         : op(op), left(left), right(right) {}
14
15     double evaluate() override {
16         double leftValue = left->evaluate();
17         double rightValue = right->evaluate();
18         if (op == '+') {
19             return leftValue + rightValue;
20         } else if (op == '-') {
21             return leftValue - rightValue;
```

```
22 -     } else if (op == '*') {
23         return leftValue * rightValue;
24 -     } else if (op == '/') {
25         if (rightValue == 0) {
26             throw std::runtime_error("Division by zero.");
27         }
28         return leftValue / rightValue;
29 -     } else {
30         throw std::runtime_error("Unknown operator.");
31     }
32 }
33 - void display(int depth) override {
34     std::cout<< std::string(depth * 2, ' ') << "Binary Operator: " << op
        << std::endl;
35     left->display(depth + 1);
36     right->display(depth + 1);
37 }
38 private:
39     char op;
40     std::shared_ptr<TreeNode> left;
41     std::shared_ptr<TreeNode> right;
```

```
42 };
43 class NumberNode : public TreeNode {
44 public:
45     NumberNode(double value) : value(value) {}
46     double evaluate() override {
47         return value;
48     }
49
50     void display(int depth) override {
51         std::cout<< std::string(depth * 2, ' ') << "Number: " << value << std
            ::endl;
52     }
53 private:
54     double value;
55 };
56 class Parser {
57 public:
58     Parser(const std::string& input) : input(input), position(0) {}
59
60     std::shared_ptr<TreeNode> parse() {
61         auto result = expression();
```

```
62 -     if (position < input.length()) {
63         throw std::runtime_error("Parsing error: Unexpected input
        .");
64     }
65     return result;
66 }
67 private:
68 - std::shared_ptr<TreeNode> expression() {
69     auto left = term();
70 -     while (position < input.length() && (input[position] == '+' ||
        input[position] == '-')) {
71         char op = input[position++];
72         auto right = term();
73         left = std::make_shared<BinaryOpNode>(op, left, right);
74     }
75     return left;
76 }
77 - std::shared_ptr<TreeNode> term() {
78     auto left = factor();
79 -     while (position < input.length() && (input[position] == '*' ||
        input[position] == '/')) {
```



```

            input[position] == '/') {
80         char op = input[position++];
81         auto right = factor();
82         left = std::make_shared<BinaryOpNode>(op, left, right);
83     }
84     return left;
85 }
86 std::shared_ptr<TreeNode> factor() {
87     if (position < input.length() && input[position] == '(') {
88         position++;
89         auto result = expression();
90         if (position < input.length() && input[position] == ')') {
91             position++;
92             return result;
93         } else {
94             throw std::runtime_error("Parsing error: Missing
                                   closing parenthesis.");
95         }
96     } else if (isdigit(input[position])) {
97         size_t end;
98         double number = std::stod(input.substr(position), &end);

```

```

99         position += end;
100         return std::make_shared<NumberNode>(number);
101     } else {
102         throw std::runtime_error("Parsing error: Unexpected
            character.");
103     }
104 }
105     const std::string& input;
106     size_t position;
107 };
108 int main() {
109     std::string input;
110     std::cout<< "Enter an arithmetic expression: ";
111     std::cin>> input;
112     try {
113         Parser parser(input);
114         std::shared_ptr<TreeNode>parseTree = parser.parse();
115         std::cout<< "Parse Tree:" << std::endl;
116         parseTree->display(0);
117
118         double result = parseTree->evaluate();

```

```

118         double result = parseTree->evaluate();
119         std::cout<< "Result: " << result << std::endl;
120     } catch (const std::exception& e) {
121         std::cerr<< "Error: " <<e.what() << std::endl;
122     }
123     return 0;
124 }

```

Output

Enter an arithmetic expression: 2+5-7*6

Parse Tree:

Binary Operator: -

Binary Operator: +

Number: 2

Number: 5

Binary Operator: *

Number: 7

Number: 6

Result: -35